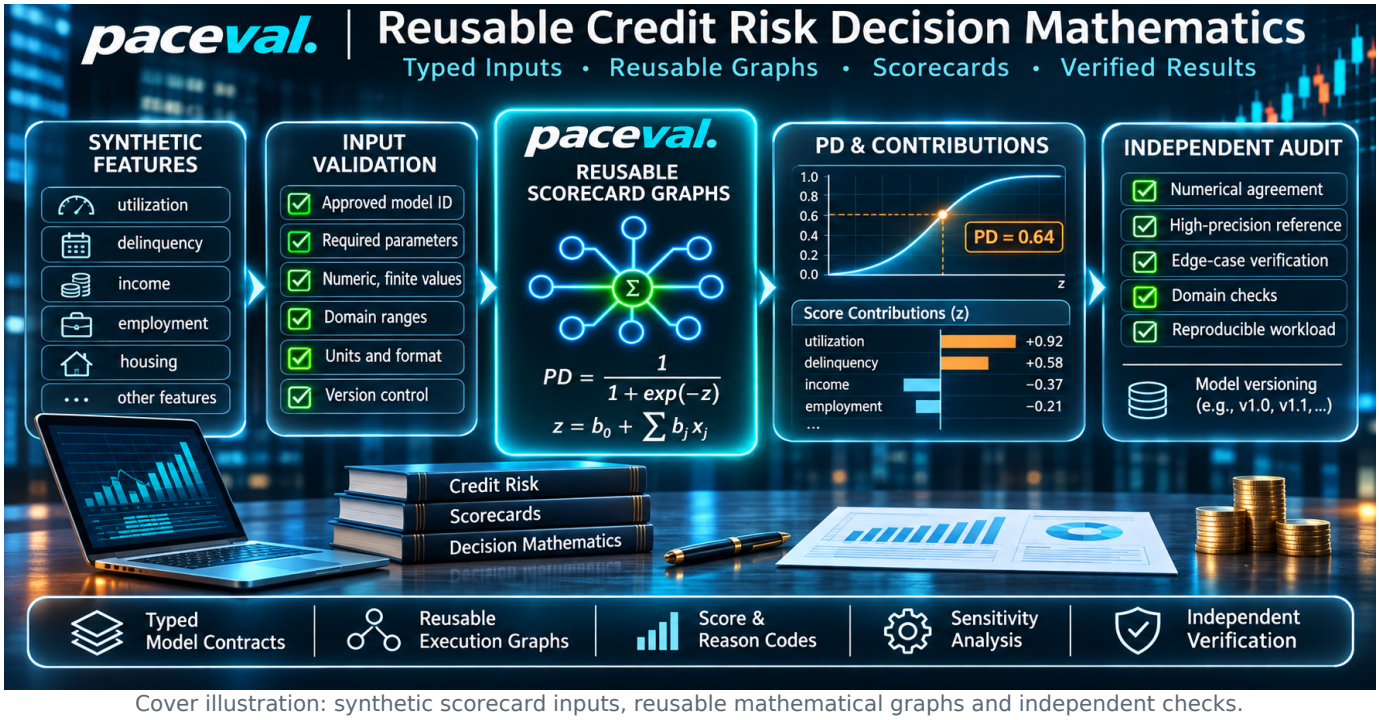
Reusable Credit Risk Decision Mathematics

An executable, independently audited paceval C++ scorecard mathematics demonstrator



Cover illustration: synthetic scorecard inputs, reusable mathematical graphs and independent checks.

Research outcome. Using the supplied paceval 4.25 C/C++ facade and a compatible Linux x86-64 engine, 100 reusable scorecard graphs executed 10,000 scalar calculations and 6,400 additional batch evaluations. All agreed with independently coded native C++ arithmetic within the declared tolerance. A separate 80-digit Python Decimal audit passed all 10,000 exported samples.

Scope. Fixed coefficients are generated synthetically and are not fitted, calibrated, or validated on borrower outcomes. All displayed logistic outputs are illustrative, not verified probabilities of default. Numerical correctness does not imply suitability for credit decisions, regulatory adverse-action notices or risk-model governance.

1. Research question and execution architecture

The original WP-018 proposal considered whether repeatedly used credit-scorecard expressions could be executed with paceval. This implementation establishes an executable mathematical use case, deterministically specified synthetic inputs, reusable formula handles and two independent arithmetic checks.

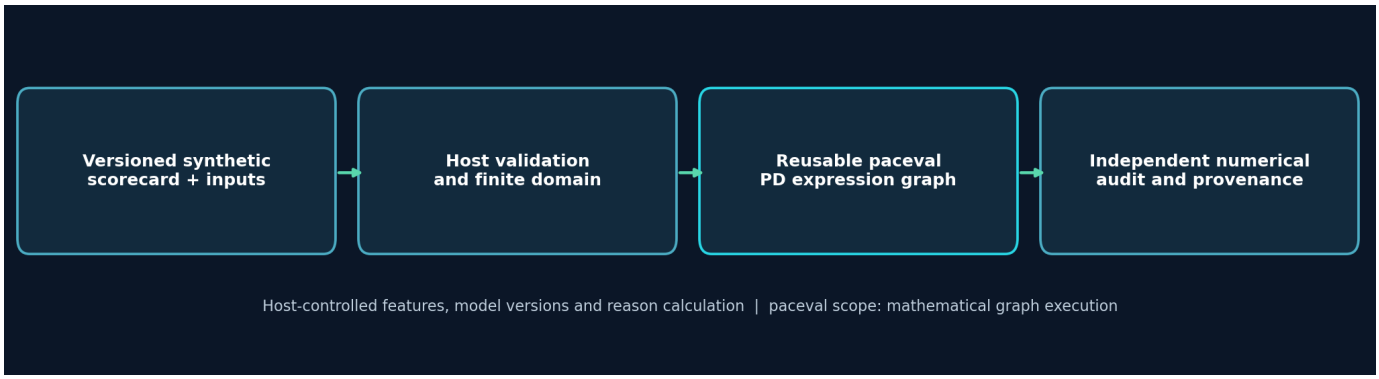


Figure 1. Execution boundaries: host-side input/model policies and independent audit are separate from graph evaluation.

The host generates bounded numerical features, maintains 100 coefficient sets, calculates additive log-odds contributions and exports traceable samples. The host creates one paceval computation handle per scorecard formula and reuses it across cases. Both the scalar API and batched API were executed with the supplied library; reason indexing is separately implemented by the host.

Evidence boundary. The actual paceval SDK path, native reference and independently specified decimal oracle were executed successfully. The numeric checks establish arithmetic feasibility only; credit-model validity and regulatory suitability are separate questions.

2. Mathematical scorecard contracts

Component	Declared mathematical meaning
Linear predictor	z = intercept + sum(j=0..4) b[j] * x[j]
Logistic transform	s(z) = 1 / (1 + exp(-z)); numerically stable native implementation
Synthetic feature domain	Five independent bounded features x[j] in [0,1]
Coefficients	100 fixed-seed synthetic intercept/coefficient sets; positive b[j]
Contribution ranking	argmax of b[j] * x[j], with first-index tie-breaking
Scenario	An illustrative single-feature intervention, not a causal counterfactual

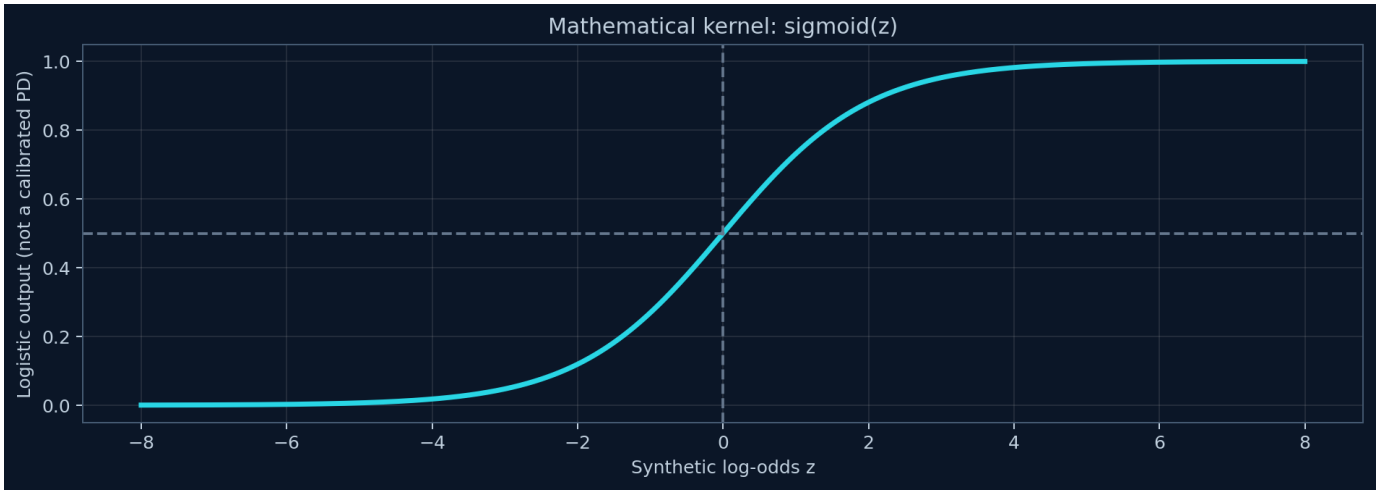


Figure 2. Mathematical logistic transform; not an empirically calibrated credit default curve.

A logistic scorecard can map log-odds to a number in (0,1). Calling that number a real-world probability of default requires labels, training and out-of-time calibration. Those requirements are outside this arithmetic experiment. The contribution ranking is an exact decomposition of this additive synthetic score only; it is not an automatically compliant adverse-action explanation.

3. Deterministic research workload

A fixed XOR-shift-style integer generator with seed 20261112 creates 100 coefficient vectors and 100 synthetic feature vectors. The same 100 vectors are reused across 100 scorecards. Two explicit boundary cases use all-zero and all-one features. The verified paceval experiment therefore contains 10,000 scalar evaluations; additionally, 64 inputs per scorecard were evaluated through the SDK batch API (6,400 outputs).

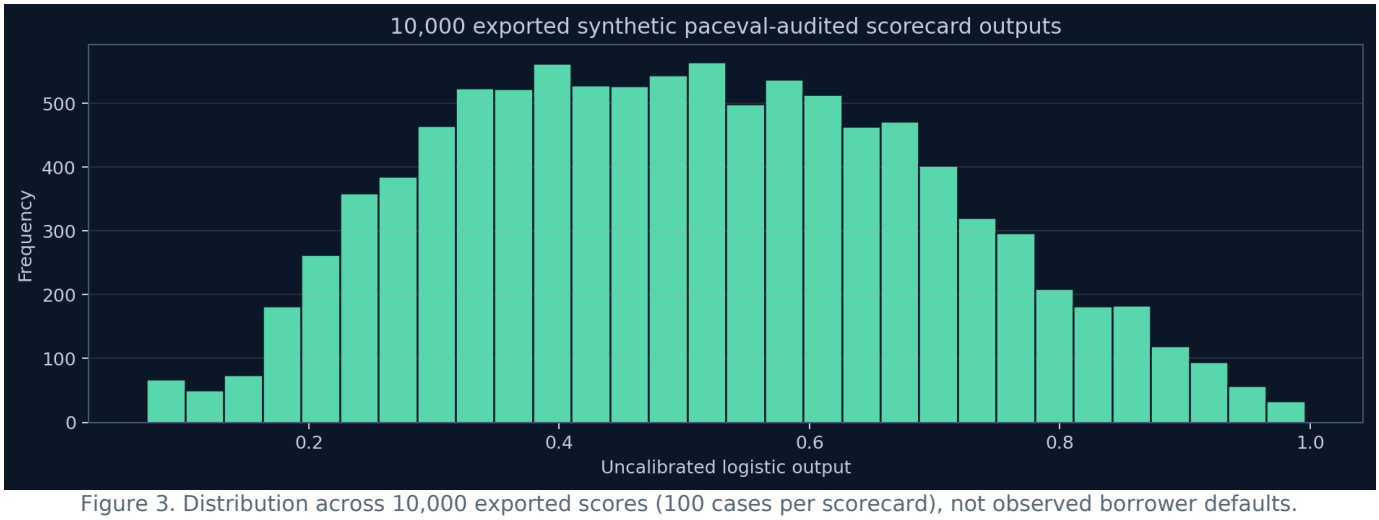


Figure 3. Distribution across 10,000 exported scores (100 cases per scorecard), not observed borrower defaults.

The export CSV contains the model identifier, all five coefficients, all five feature values, intercept, linear predictor, logistic output and strongest positive log-odds contribution index. This makes the exported arithmetic independently reproducible. It does not contain borrower records, protected attributes, observed outcomes or actual credit decisions.

4. paceval execution and independent numerical audit

Verification	Observed result
SDK-enabled CTest	1 / 1 passed
Reusable paceval graphs	100 created and reused
paceval scalar versus native	10,000 / 10,000 passed
paceval batch versus native	6,400 / 6,400 passed
Independent Decimal audit	10,000 / 10,000 passed
Decimal working precision	80 significant decimal digits
Maximum exported-score discrepancy	1.9810e-16
Numerical thresholds	SDK/native: 1e-10; Decimal/export: 1e-12



Figure 4. Independent 80-digit reconstruction of the exported synthetic arithmetic outputs.

The Decimal audit reconstructs the linear predictor and logistic transform from the printed decimal coefficients and feature values, then verifies the recorded score and contribution ranking. The maximum deviation reflects binary64 versus decimal evaluation and finite export precision. It does not constitute an exhaustive proof of correctness over all possible inputs.

5. Sensitivity analysis and contribution tracing

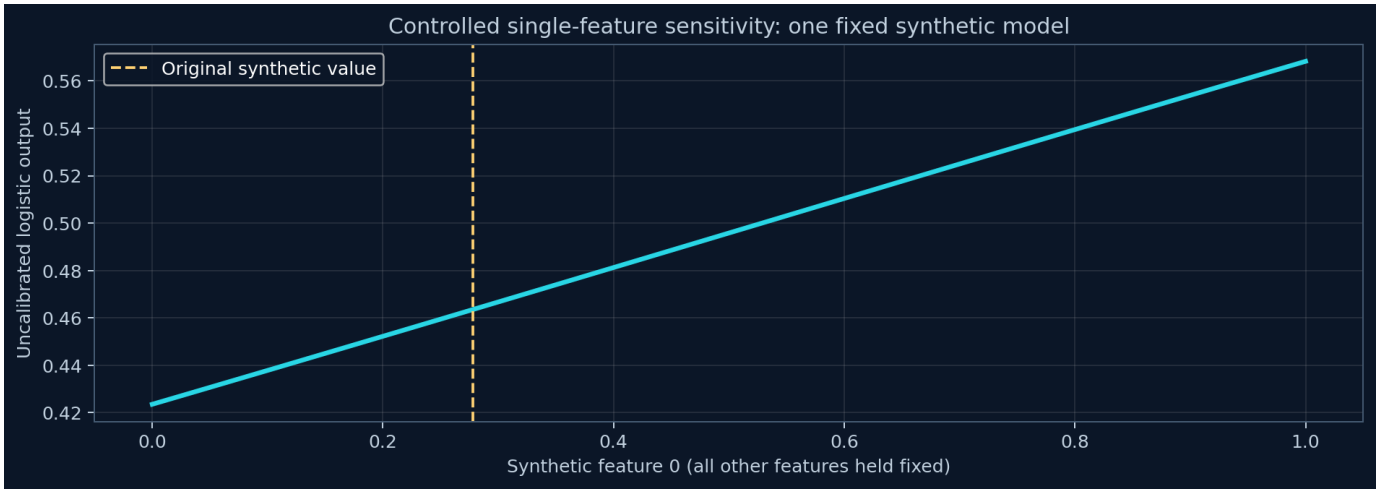


Figure 5. One-feature sensitivity in a fixed synthetic scorecard; remaining inputs and coefficients do not change.

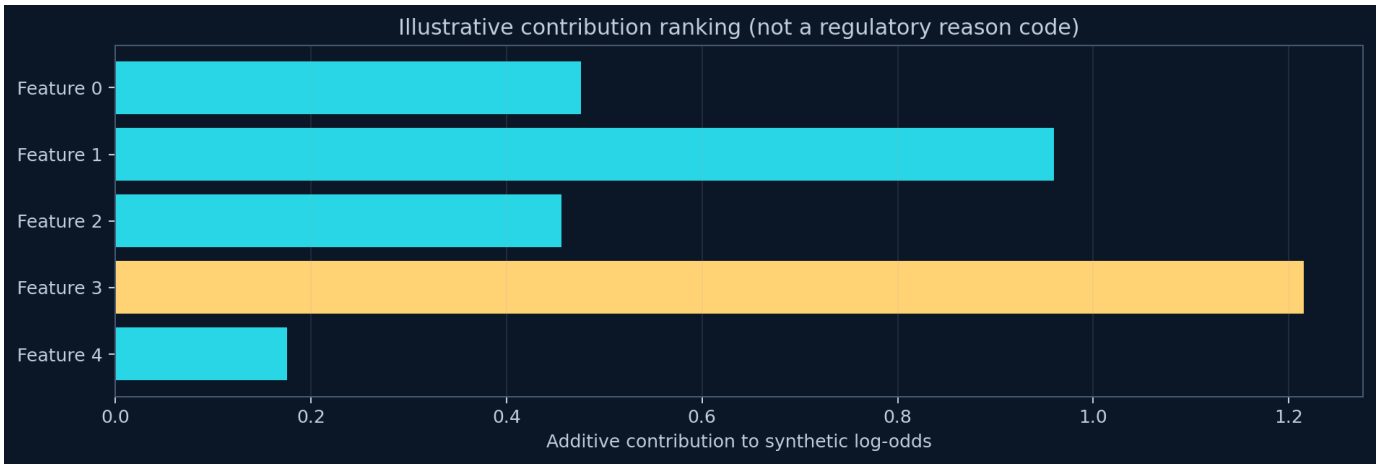


Figure 6. Additive log-odds decomposition with the strongest positive synthetic contribution highlighted.

Sensitivity is measured by direct reevaluation of the logistic expression as one normalized feature varies over [0,1]. These are hypothetical numerical perturbations, not causal interventions. Because coefficients are all positive in this pilot, an increased feature cannot reduce the logistic output; this restriction is a property of the synthetic generator, not of real credit data.

6. Reusable expression graphs and model traceability

Each of the 100 coefficient sets is treated as a different research formula version. A paceval computation object was created once per formula and reused for new synthetic input vectors. Each handle is checked and disposed of by the host. A production system would require persistent registry metadata, stable schema signatures and explicit invalidation/promotion procedures beyond this program.

The study intentionally addresses successful mathematical graph execution, arithmetic agreement and model traceability. The tested computation contracts are independent of processor-specific timing. No claims about financial accuracy, model discrimination or deployment readiness follow from these tests.

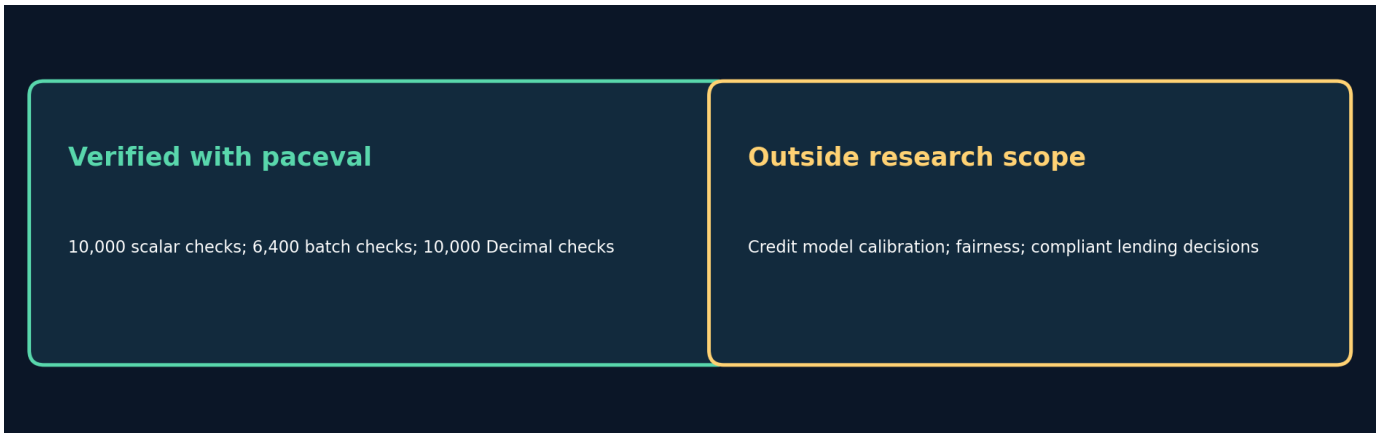


Figure 7. Verified mathematical execution and numerical checks contrasted with untested financial-model suitability.

The project makes no claims concerning SQL equivalence, scorecard calibration, AUC, fairness, causal attribution or regulatory approvals. These are outside the released mathematical feasibility experiment.

7. Reproducibility, implementation and governance

Build. Run **cmake -S src -B build -DPACEVAL_RUNTIME=/absolute/path/to/libpaceval_linux_sharedLIB.so** followed by **cmake --build build** and **ctest --test-dir build --output-on-failure**. Run **./build/credit_scorecard --models 100 --cases 100 --out reproduced.csv**, then run the bundled 80-digit high-precision audit against that CSV.

Responsibility	Implemented component
Synthetic scorecards and native reference	src/credit_scorecard.cpp
Reusable paceval graph execution	src/paceval_main.cpp plus compatible external shared engine
Numerical high-precision oracle	src/high_precision_reference.py
Scientific figures	src/make_figures.py
Paper rendering	src/make_whitepaper.py
Synthetic sample records	examples/scorecard_samples.csv

The package includes the supplied paceval_main.h and paceval_main.cpp (public SDK 4.25 facade), with the relative include adjusted for the local src/ directory. The engine shared library is an external runtime dependency and is intentionally not redistributed. The original SDK source copyright and AGPL notices are retained. The SDK-enabled source, including scalar and batch APIs, was compiled and executed with the supplied compatible Linux runtime.

Governance. EBA loan-origination guidelines address creditworthiness assessment and monitoring. Some AI systems used for evaluating natural-person creditworthiness fall within Annex III of Regulation (EU) 2024/1689, subject to the applicable legal provisions. US Regulation B expects specific, accurate principal adverse-action reasons. No compliance is claimed for this arithmetic research project.

8. External references and conclusions

1. paceval mathematical engine and C/C++ API
https://paceval.com/api/

2. paceval public sources and examples
https://github.com/paceval/paceval

3. EBA loan origination and monitoring guidelines, EBA/GL/2020/06
https://www.eba.europa.eu/activities/single-rulebook/regulatory-activities/credit-risk/guidelines-loan-origination-and-monitoring

4. EU Artificial Intelligence Act, Regulation (EU) 2024/1689, Annex III
https://eur-lex.europa.eu/eli/reg/2024/1689/oj/eng

5. US CFPB, Regulation B, 12 CFR 1002.9
https://www.consumerfinance.gov/rules-policy/regulations/1002/9/

6. CFPB circular on adverse-action reasons for complex algorithms
https://www.consumerfinance.gov/compliance/circulars/circular-2022-03-adverse-action-notification-requirements-in-connection-with-credit-decisions-based-on-complex-algorithms/

7. UCI Statlog German Credit dataset (not included or analyzed)
https://archive.ics.uci.edu/dataset/144/statlog+german+credit+data

Conclusion. The defined credit-scorecard arithmetic use case has been implemented and executed successfully with the supplied paceval engine. In 100 reusable graphs, all 10,000 scalar and 6,400 batch evaluations agreed with independent native C++ arithmetic within the specified tolerance; an additional 80-digit Decimal audit verified all 10,000 exported scores. The outcome demonstrates numerical feasibility, not calibrated credit-default predictions, regulatory readiness or customer adoption.